

循序渐进，学习开发 xv6-riscv

第 17 章 进程的睡眠与唤醒

汪辰



目录

01

基础框架

02

系统调用 pause & uptime

03

系统调用 read

04

用户态 shell



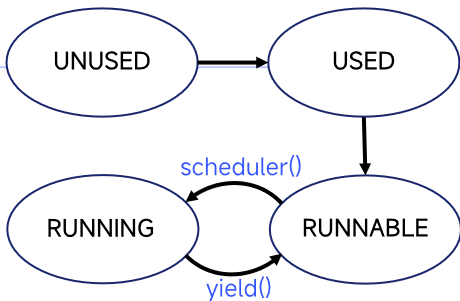
01

基础框架



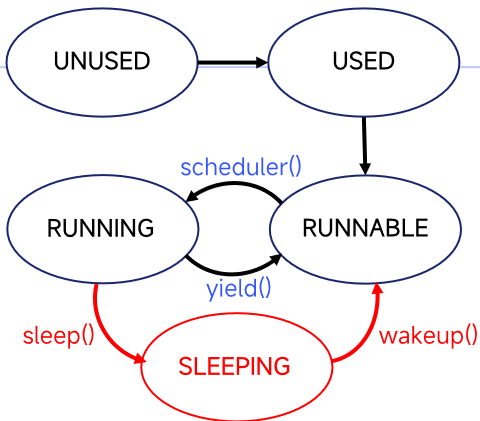
任务状态迁移

- 任务调度



任务状态迁移

- 任务调度
- 睡眠与唤醒



Sleep & Wakeup API 设计 (v1)

void sleep(void *chan);

消费者

```
if (执行条件 == false) {  
    sleep(&chan);  
}  
// 执行 (do something)
```

chan (等待通道)

void wakeup(void *chan);

生产者

```
执行条件 = true;  
wakeup(&chan);
```

Sleep & Wakeup API 设计 (v1)

void sleep(void *chan);

消费者

```
if (执行条件 == false) {  
    sleep(&chan);  
}  
// 执行 (do something)
```

chan (等待通道)

```
diff --git a/kernel/proc.h b/kernel/proc.h  
index 67cffe..e3b51b1 100644  
--- a/kernel/proc.h  
+++ b/kernel/proc.h  
@@ -87,6 +87,7 @@ struct proc {  
  
    // p->lock must be held when  
    enum procstate state;  
+    void *chan;  
    int pid;
```

void wakeup(void *chan);

生产者

```
执行条件 = true;  
wakeup(&chan);
```

```
void sleep(void *chan)  
{  
    struct proc *p = myproc();  
    acquire(&p->lock); //DOC: sleeplock1  
  
    p->chan = chan;  
    p->state = SLEEPING;  
    sched();  
    p->chan = 0;  
    release(&p->lock);  
}
```

```
void wakeup(void *chan)  
{  
    struct proc *p;  
    for(p = proc; p < &proc[NPROC]; p++) {  
        if(p != myproc()){  
            acquire(&p->lock);  
            if(p->state == SLEEPING && p->chan == chan) {  
                p->state = RUNNABLE;  
            }  
            release(&p->lock);  
        }  
    }  
}
```

Sleep & Wakeup API 设计 (v1)

void sleep(void *chan);

消费者

```
if (执行条件 == false) {  
    sleep(&chan);  
}  
// 执行 (do something)
```

chan (等待通道)

void wakeup(void *chan);

生产者

```
执行条件 = true;  
wakeup(&chan);
```

```
diff --git a/kernel/proc.h b/kernel/proc.h  
index 67cffe..e3b51b1 100644  
--- a/kernel/proc.h  
+++ b/kernel/proc.h  
@@ -87,6 +87,7 @@ struct proc {  
  
    // p->lock must be held when  
    enum procstate state;  
+ void *chan;  
    int pid;
```

```
void sleep(void *chan,  
{  
    struct proc *p = myproc();  
    acquire(&p->lock); //DOC: sleep1  
  
    p->chan = chan;  
    p->state = SLEEPING;  
    sched();  
    p->chan = 0;  
    release(&p->lock);  
}
```

丢失唤醒

```
void wakeup(void *chan)  
{  
    struct proc *p;  
    for(p = proc; p < &proc[NPROC]; p++) {  
        if(p != myproc()) {  
            acquire(&p->lock);  
            if(p->state == SLEEPING && p->chan == chan) {  
                p->state = RUNNABLE;  
            }  
            release(&p->lock);  
        }  
    }  
}
```

Sleep & Wakeup API 设计 (v2)

void sleep(void *chan, struct spinlock *lk);

void wakeup(void *chan);

消费者

chan (等待通道)

生产者

```
acquire(&cond_lock);
if (执行条件 == false) {
    sleep(&chan, &cond_lock);
}
// 执行 (do something)
release(&cond_lock);
```

```
diff --git a/kernel/proc.h b/kernel/proc.h
index 67cffe..e3b51b1 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -87,6 +87,7 @@ struct proc {
    // p->lock must be held when
    enum procstate state;
+   void *chan;
    int pid;
```

```
acquire(&cond_lock);
执行条件 = true;
wakeup(&chan);
release(&cond_lock);
```

```
void sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();
    acquire(&p->lock); //DOC: sleeplock1
    release(lk);
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    p->chan = 0;
    release(&p->lock);
    acquire(lk);
}
```

```
void wakeup(void *chan)
{
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                p->state = RUNNABLE;
            }
            release(&p->lock);
        }
    }
}
```

Sleep & Wakeup API 设计 (v2)

void sleep(void *chan, struct spinlock *lk);

void wakeup(void *chan);

消费者

chan (等待通道)

生产者

```
acquire(&cond_lock);
if (执行条件 == false) {
    sleep(&chan, &cond_lock);
}
// 执行 (do something)
release(&cond_lock);
```

```
diff --git a/kernel/proc.h b/kernel/proc.h
index 67cffe..e3b51b1 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -87,6 +87,7 @@ struct proc {
    // p->lock must be held when
    enum procstate state;
+   void *chan;
    int pid;
```

```
acquire(&cond_lock);
执行条件 = true;
wakeup(&chan);
release(&cond_lock);
```

虚假唤醒

```
void sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();
    acquire(&p->lock); //DOC: sleep-acquire
    release(lk);
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    p->chan = 0;
    release(&p->lock);
    acquire(lk);
}
```

```
void wakeup(void *chan)
{
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                p->state = RUNNABLE;
            }
            release(&p->lock);
        }
    }
}
```

Sleep & Wakeup API 设计 (v3)

void sleep(void *chan, struct spinlock *lk);

void wakeup(void *chan);

消费者

chan (等待通道)

生产者

```
acquire(&cond_lock);
while (执行条件 == false) {
    sleep(&chan, &cond_lock);
}
// 执行 (do something)
release(&cond_lock);
```

```
diff --git a/kernel/proc.h b/kernel/proc.h
index 67cffe..e3b51b1 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -87,6 +87,7 @@ struct proc {
    // p->lock must be held when
    enum procstate state;
+   void *chan;
    int pid;
```

```
acquire(&cond_lock);
执行条件 = true;
wakeup(&chan);
release(&cond_lock);
```

```
void sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();
    acquire(&p->lock); //DOC: sleeplock1
    release(lk);
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    p->chan = 0;
    release(&p->lock);
    acquire(lk);
}
```

```
void wakeup(void *chan)
{
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                p->state = RUNNABLE;
            }
            release(&p->lock);
        }
    }
}
```

Sleep & Wakeup API 设计 (Final)

void sleep(void *chan, struct spinlock *lk);

void wakeup(void *chan);

消费者

chan (等待通道)

生产者

```
acquire(&cond_lock);
while (执行条件 == false) {
    sleep(&chan, &cond_lock);
}
// 执行 (do something)
release(&cond_lock);
```

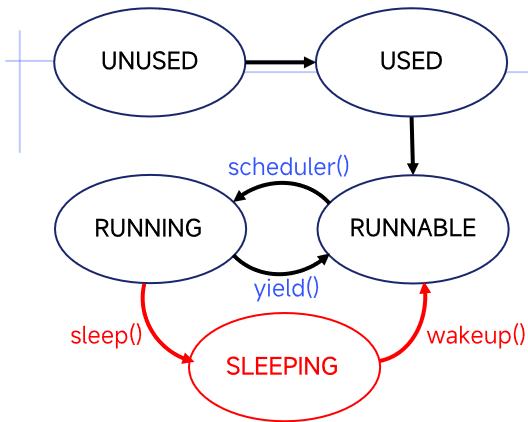
```
diff --git a/kernel/proc.h b/kernel/proc.h
index 67cffe..e3b51b1 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -87,6 +87,7 @@ struct proc {
    // p->lock must be held when
    enum procstate state;
+   void *chan;
    int pid;
```

```
acquire(&cond_lock);
执行条件 = true;
wakeup(&chan);
release(&cond_lock);
```

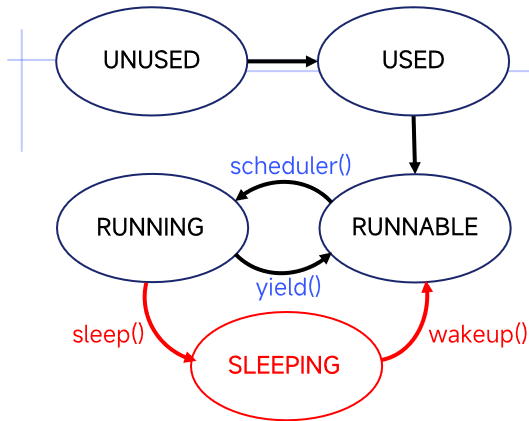
```
void sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();
    acquire(&p->lock); //DOC: sleeplock1
    release(lk);
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    p->chan = 0;
    release(&p->lock);
    acquire(lk);
}
```

```
void wakeup(void *chan)
{
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                p->state = RUNNABLE;
            }
            release(&p->lock);
        }
    }
}
```

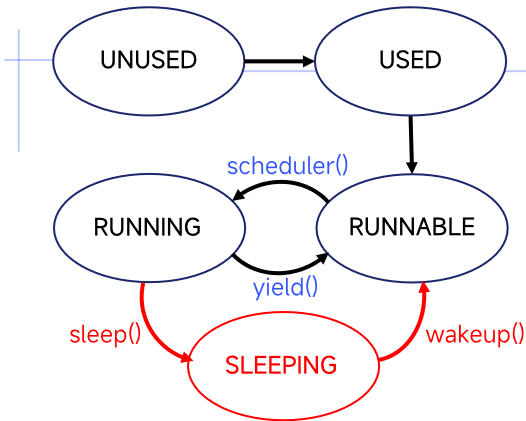
Scheduler 的改进



Scheduler 的改进



```
diff --git a/kernel/proc.c b/kernel/proc.c
index 54adb30..e6b9310 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
.....
@@ -268,6 +269,15 @@ scheduler(void)
{
    c->proc = 0;
    for(;;){
+       // The most recent process to run may have had interrupts
+       // turned off; enable them to avoid a deadlock if all
+       // processes are waiting. Then turn them back off
+       // to avoid a possible race between an interrupt
+       // and wfi.
+       intr_on();
+       intr_off();
+
+       int found = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
@@ -281,9 +291,14 @@ scheduler(void)
                // Process is done running for now.
                // It should have changed its p->state before coming back.
                c->proc = 0;
                found = 1;
            }
            release(&p->lock);
        }
        if(found == 0) {
+           // nothing to run; stop running on this core until an interrupt.
+           asm volatile("wfi");
        }
    }
}
```

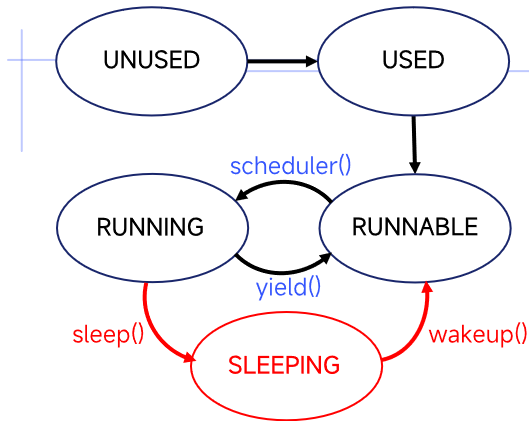


```
c->proc = 0;
for(;;){
+ // The most recent process to run may have had interrupts
+ // turned off; enable them to avoid a deadlock if all
+ // processes are waiting. Then turn them back off
+ // to avoid a possible race between an interrupt
+ // and wfi.
+ intr_on();
+ intr_off();
}
```

3

- WFI (Wait For Interrupt) 是 RISC-V 中的一个重要指令，用于让硬件核心 (hart) 进入低功耗的等待状态 (stall)，直到有中断需要处理。
- 唤醒条件与本地中断使能有关，但与全局中断使能无关。
- 如果唤醒事件是一个待处理的中断，处理器会正常进入该中断的陷阱 (Trap) 处理程序，程序计数器 PC 指向 WFI 的下一条指令 (PC+4) 。
- 如果唤醒原因是其他事件 (非中断)，则处理器会直接从 WFI 指令后的下一条指令继续执行。

Scheduler 的改进



```
diff --git a/kernel/proc.c b/kernel/proc.c
index 54adb30..e6b9310 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
.....
@@ -268,6 +269,15 @@ scheduler(void)
```

```
c->proc = 0;
for(;;){
+ // The most recent process to run may have had interrupts
+ // turned off; enable them to avoid a deadlock if all
+ // processes are waiting. Then turn them back off
+ // to avoid a possible race between an interrupt
+ // and wfi.
```

```
+ intr_on();
+ intr_off();
```

```
+ int found = 0;
+ for(p = proc; p < &proc[NPROC]; p++) {
+   acquire(&p->lock);
+   if(p->state == RUNNABLE)
@@ -281,9 +291,14 @@ scheduler(void)
+ // Process is done running
+ // It should have changed state
+ c->proc = 0;
+ found = 1;
```

```
+ }
+ release(&p->lock);
+ }
+ if(found == 0) {
+ // nothing to run; stop
+ asm volatile("wfi");
+ }
+ }
```

- `intr_on()` 时会尝试将 WFI 期间累积的中断捞出来处理
- 关闭中断，确保接下来的遍历 `proc` 数组操作不受中断影响

- WFI (Wait For Interrupt) 是 RISC-V 中的一个重要指令，用于让硬件核心 (hart) 进入低功耗的等待状态 (stall)，直到有中断需要处理。

- 唤醒条件与本地中断使能有关，但与全局中断使能无关。

- 如果唤醒事件是一个待处理的中断，处理器会正常进入该中断的陷阱 (Trap) 处理程序，程序计数器 PC 指向 WFI 的下一条指令 (PC+4)。

- 如果唤醒原因是其他事件 (非中断)，则处理器会直接从 WFI 指令后的下一条指令继续执行。



02



系统调用

pause & uptime

系统调用 pause

```
sleep(3)                                Library Functions Manual    sleep(3)
NAME
    sleep - sleep for a specified number of seconds
LIBRARY
    Standard C library (libc, -lc)
SYNOPSIS
    #include <unistd.h>

    unsigned int sleep(unsigned int seconds);
```

man 3 sleep

系统调用 pause

`sleep(3)` Library Functions Manual `sleep(3)`

NAME

`sleep` - sleep for a specified number of seconds

LIBRARY

Standard C library (`libc`, `-lc`)

SYNOPSIS

```
#include <unistd.h>
```

```
unsigned int sleep(unsigned int seconds);
```

user/user.h

```
int pause(int ticks);
```

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index f0b9335..afcde30 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -47,12 +47,16 @@ argaddr(int n, uint64 *ip)

// Prototypes for the functions that handle system calls.
extern uint64 sys_getpid(void);
+extern uint64 sys_pause(void);
extern uint64 sys_write(void);

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_getpid] sys_getpid,
+[SYS_pause] sys_pause,
[SYS_write] sys_write,
};
```

```
uint64 sys_pause(void)
{
    int n;
    uint ticks0;

    argint(0, &n);
    if(n < 0)
        n = 0;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    return 0;
}
```

```
struct spinlock tickslock;
uint ticks;
```

```
void clockintr()
{
    if(cpuid() == 0){
        acquire(&tickslock);
        ticks++;
        wakeup(&ticks);
        release(&tickslock);
    }
    // .....
```

系统调用 uptime

```
time(2)                                System Calls Manual                                time(2)

NAME
    time - get time in seconds

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <time.h>

    time_t time(time_t *_Nullable tloc);
```

man 2 time

系统调用 uptime

```
time(2)                                     System Calls Manual                                     time(2)

NAME
    time - get time in seconds

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <time.h>

    time_t time(time_t *_Nullable tloc);
```

user/user.h

int uptime(void);

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index f0b9335..afcd30 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -47,12 +47,16 @@ argaddr(int n, uint64 *ip)

// Prototypes for the functions that handle system calls.
extern uint64 sys_getpid(void);
extern uint64 sys_pause(void);
+extern uint64 sys_uptime(void);
extern uint64 sys_write(void);

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_getpid] sys_getpid,
[SYS_pause] sys_pause,
+[SYS_uptime] sys_uptime,
[SYS_write] sys_write,
};
```

```
uint64 sys_uptime(void)
{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}
```

测试系统调用 `pause` 和 `uptime`

```
int
main(void)
{
    int ticks1, ticks2;
    int pid = getpid();
    printf("hello world! my pid is %d\n", pid);

    printf("--> Sleeping 2s ..... \n");
    ticks1 = uptime();
    // sleep is a system call that sleeps for a specified number of ticks
    // every tick is 1000000, about a tenth of a second(0.1s).
    pause(20); // 2 seconds
    ticks2 = uptime();
    printf("<-- Woke-up! Elapsed ticks = %d.\n", ticks2 - ticks1);

    return 0;
}
```

```
$ make gemu
qemu-system-riscv64 -machine virt -bios none -kernel
kernel/kernel -m 128M -smp 3 -nographic
```

xv6 kernel is booting

hart 2 starting

hart 1 starting

hello world! my pid is 1

--> Sleeping 2s

<-- Woke-up! Elapsed ticks = 20.



03

系统调用 read

系统调用 read

```
read(2)                                System Calls Manual                                read(2)

NAME
    read - read from a file descriptor

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <unistd.h>

    ssize_t read(int fd, void buf[.count], size_t count);
```

man 2 read

系统调用 read

read(2) System Calls Manual read(2)

NAME

read - read from a file descriptor

LIBRARY

Standard C library

SYNOPSIS

#include <unistd.h>

ssize_t read(int fd, void buf[.count], size_t count);

user/user.h

int read(int fd, void* buf, int count);

系统调用 read

read(2) System Calls Manual read(2)

NAME

read - read from a file descriptor

LIBRARY

Standard C library

SYNOPSIS

```
#include <unistd.h>
```

```
ssize_t read(int fd, void buf[.count], size_t count);
```

user/user.h

int read(int fd, void* buf, int count);

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index afcde30..e0d73cf 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -46,6 +46,7 @@ argaddr(int n, uint64 *ip)
 }

// Prototypes for the functions that handle system calls.
+extern uint64 sys_read(void);
extern uint64 sys_getpid(void);
extern uint64 sys_pause(void);
extern uint64 sys_uptime(void);
@@ -54,6 +55,7 @@ extern uint64 sys_write(void);
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
+ [SYS_read] sys_read,
 [SYS_getpid] sys_getpid,
 [SYS_pause] sys_pause,
 [SYS_uptime] sys_uptime,
```

```
diff --git a/kernel/sysfile.c b/kernel/sysfile.c
index c250638..c11e886 100644
--- a/kernel/sysfile.c
+++ b/kernel/sysfile.c
@@ -11,8 +11,22 @@
#include "spinlock.h"
#include "proc.h"

+extern int consoleread(int user_dst, uint64 dst, int n);
extern int consolewrite(int user_src, uint64 src, int n);

+uint64
+sys_read(void)
+{
+ int n;
+ uint64 p;
+
+ // ignore arg 0 (fd)
+ argaddr(1, &p);
+ argint(2, &n);
+
+ return consoleread(1, p, n);
+}
```

系统调用 read

```
uint64 sys_read(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consoleread(1, p, n);
}
```



系统调用 read

```
uint64 sys_read(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consolaread(1, p, n);
}
```

```
diff --git a/kernel/console.c b/kernel/console.c
index a15fd5e..e6f3792 100644
--- a/kernel/console.c
+++ b/kernel/console.c
.....
int
-consolaread(uint64 dst, int n)
+consolaread(int user_dst, uint64 dst, int n)
{
.....
    acquire(&cons.lock);
    while(n > 0){
        // wait until interrupt handler has put some
        // input into cons.buffer.
        release(&cons.lock);
        while(cons.r == cons.w){
            asm volatile ("" : : : "memory"); // Prevent
+            sleep(&cons.r, &cons.lock);
        }
        acquire(&cons.lock);

        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
.....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        - *((char *)dst) = cbuf;
+         if(either_copyout(user_dst, dst, &cbuf, 1) == -1)
+             break;
.....
    }
    release(&cons.lock);

    return target - n;
}
```

```
diff --git a/kernel/console.c b/kernel/console.c
index a15fd5e..e6f3792 100644
--- a/kernel/console.c
+++ b/kernel/console.c
@@ -166,6 +168,7 @@ consoleintr(int c)
{
    acquire(&cons.lock);

    switch(c){
        .....
        default:
            if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                .....
                if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                    // wake up consolaread() if a whole line (or end-of-file)
                    // has arrived.
                    cons.w = cons.e;
+                    wakeup(&cons.r);
                }
            }
            break;
        }

    release(&cons.lock);
}
```

系统调用 read

```
uint64 sys_read(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consleread(1, p, n);
}
```

```
diff --git a/kernel/console.c b/kernel/console.c
index a15fd5e..e6f3792 100644
--- a/kernel/console.c
+++ b/kernel/console.c
.....
int
- consleread(uint64 dst, int n)
+ consleread(int user_dst, uint64 dst, int n)
{
.....
    acquire(&cons.lock);
    while(n > 0){
        // wait until interrupt handler has put some
        // input into cons.buffer.
        release(&cons.lock);
        while(cons.r == cons.w){
-         asm volatile ("": : : "memory"); // Prevent
+         sleep(&cons.r, &cons.lock);
        }
        acquire(&cons.lock);

        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
.....
        // copy the input byte to the user-space buffer.
        cbuf = c;
-         *((char *)dst) = cbuf;
+         if(either_copyout(user_dst, dst, &cbuf, 1) == -1)
+             break;
.....
    }
    release(&cons.lock);

    return target - n;
}
```

```
diff --git a/kernel/console.c b/kernel/console.c
index a15fd5e..e6f3792 100644
--- a/kernel/console.c
+++ b/kernel/console.c
@@ -166,6 +168,7 @@ consoleintr(int c)
{
    acquire(&cons.lock);

    switch(c){
.....
    default:
        if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
            .....
            if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                // wake up consleread() if a whole line (or end-of-file)
                // has arrived.
                cons.w = cons.e;
+                wakeup(&cons.r);
            }
        }
        break;
    }

    release(&cons.lock);
}

int either_copyout(int user_dst, uint64 dst, void *src, uint64 len)
{
    struct proc *p = myproc();
    if(user_dst){
        return copyout(p->pagetable, dst, src, len);
    } else {
        memmove((char *)dst, src, len);
        return 0;
    }
}
```

改进系统调用 write

```
uint64 sys_write(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consolewrite(1, p, n);
}
```

改进系统调用 write

```
uint64 sys_write(void)
{
    int consolewrite(int user_src, uint64 src, int n)
    {
        char buf[32]; // move batches from user space to uart.
        int i = 0;

        while(i < n){
            int nn = sizeof(buf);
            if(nn > n - i)
                nn = n - i;
            if(either_copyin(buf, user_src, src+i, nn) == -1)
                break;
            uartwrite(buf, nn);
            i += nn;
        }

        return i;
    }
}
```

改进系统调用 write

```
uint64 sys_write(void)
```

```
{
    int consolewrite(int use
    {
        uint char buf[32]; // move
        int i = 0;

        // i
        arga while(i < n){
        argi     int nn = sizeof(buf)
        retu     if(nn > n - i)
                nn = n - i;
                if(either_copyin(buf
                    break;
                    uartwrite(buf, nn);
                    i += nn;
                }

                return i;
            }
}
```

```
diff --git a/kernel/uart.c b/kernel/uart.c
```

```
index 20a992d..248b9e4 100644
```

```
--- a/kernel/uart.c
```

```
+++ b/kernel/uart.c
```

```
@@ -40,6 +40,7 @@
```

```
.....
```

```
+static int tx_chan;           // &tx_chan is the "wait channel"
```

```
.....
```

```
@@ -82,14 +83,12 @@ uartwrite(char buf[], int n)
```

```
{
```

```
    acquire(&tx_lock);
```

```
    int i = 0;
```

```
    while(i < n){
```

```
-    release(&tx_lock);
```

```
    while(tx_busy != 0){
```

```
        // wait for a UART transmit-complete interrupt
```

```
        // to set tx_busy to 0.
```

```
-    asm volatile ("" : : "memory"); // Preventin
```

```
-    }
```

```
-    acquire(&tx_lock);
```

```
-
```

```
-
```

```
+    sleep(&tx_chan, &tx_lock);
```

```
+    }
```

```
+    WriteReg(THR, buf[i]);
```

```
    i += 1;
```

```
    tx_busy = 1;
```

```
    }
```

```
    release(&tx_lock);
```

```
.....
```

```
diff --git a/kernel/uart.c b/kernel/uart.c
```

```
index 20a992d..248b9e4 100644
```

```
--- a/kernel/uart.c
```

```
+++ b/kernel/uart.c
```

```
.....
```

```
@@ -148,6 +147,7 @@ uartintr(void)
```

```
.....
```

```
    acquire(&tx_lock);
```

```
    if(ReadReg(LSR) & LSR_TX_IDLE){
```

```
        // UART finished transmitting; wake up sending thread.
```

```
        tx_busy = 0;
```

```
+    wakeup(&tx_chan);
```

```
    }
```

```
    release(&tx_lock);
```

一个简单的测试

user/init.c

```
#include "user/user.h"

int
main(void)
{
    char c;

    for(;;) {
        // read will block until there is a enter,
        // or EOF or console buffer is full.
        read(0, &c, 1);
        printf("->");
        printf("%c", c);
    }
}
```

一个简单的测试

user/init.c

```
#include "user/user.h"

int
main(void)
{
    char c;

    for(;;) {
        // read will block until
        // or EOF or console buffer
        read(0, &c, 1);
        printf("->");
        printf("%c", c);
    }
}
```

- 进入 for 循环后 read 函数会阻塞 (sleep) 直到输入遇到“回车”/“EOF”或者内部控制台缓存满才会退出 (被唤醒)。
- 控制台上第一行输出是回显 (在键盘输入时产生), 包括最后一个回车。
- 控制台上的第二行输出是循环过程中 read 函数返回后的打印的结果, 一次读取内部缓存中的一个字符, 包括最后的回车。

```
$ make qemu
qemu-system-riscv64 -s -S -nographic -bios none -kernel
kernel/kernel -m 128M -device virtio-blk-device,drive=hd0 -device virtio-net-device,netdev=net0 -device virtio-serial-device
xv6 kernel is booting
hart 1 starting
hart 2 starting
123456
->1->2->3->4->5->6->
```



04

用户态 Shell

用户态 Shell

```
diff --git a/Makefile b/Makefile
index efbffd6..18ddd78 100644
--- a/Makefile
+++ b/Makefile
@@ -96,8 +96,10 @@ $K/initcode.c: $U/initcode
.....
-$U/initcode: $U/start.o $U/init.o $U/usys.o $U/printf.o
-      $(LD) $(LDFLAGS) -N -e _start -Ttext 0 -o $U/initcode.out $U/start.o $U/init.o $U/usys.o $U/printf.o
+ULIB = $U/ulib.o $U/usys.o $U/printf.o
+
+$U/initcode: $U/start.o $U/init.o $(ULIB)
+      $(LD) $(LDFLAGS) -N -e _start -Ttext 0 -o $U/initcode.out $U/start.o $U/init.o $(ULIB)
+      $(OBJCOPY) -S -O binary $U/initcode.out $U/initcode
+      $(OBJDUMP) -S $U/initcode.out > $U/initcode.asm
```

用户态 Shell

.globl _start
.globl main
_start:
call main
stop:
j stop

strlen()
memset()
gets()

.global read
read:
li a7, SYS_read
ecall
ret
.global write
.....

vprintf()
fprintf()
printf()

libc

```
.....  
-$U/initcode: $U/start.o $U/init.o $U/usys.o $U/printf.o  
- $(LD) $(LDFLAGS) -N -e _start -Ttext 0 -o $U/initcode.out $U/start.o $U/init.o $U/usys.o $U/printf.o  
+ULIB = $U/ulib.o $U/usys.o $U/printf.o  
+ $U/initcode: $U/start.o $U/init.o $(ULIB)  
+ $(LD) $(LDFLAGS) -N -e _start -Ttext 0 -o $U/initcode.out $U/start.o $U/init.o $(ULIB)  
$(OBJCOPY) -S -O binary $U/initcode.out $U/initcode  
$(OBJDUMP) -S $U/initcode.out > $U/initcode.asm
```

用户态 Shell

.globl _start
.globl main
_start:
 call main
stop:
 j stop

strlen()
memset()
gets()

.global read
read:
 li a7, SYS_read
 ecall
 ret
.global write
.....

vprintf()
fprintf()
printf()

libc

```
.....  
- $U/initcode: $U/start.o $U/init.o $U/usys.o $U/printf.o  
- $(LD) $(LDFLAGS) -N -e _start -Ttext 0 -o $U/initcode.out $U/initcode.o $U/usys.o $U/printf.o  
+ULIB = $U/ulib.o $U/usys.o $U/printf.o  
+ $U/initcode: $U/start.o $U/init.o $(ULIB)  
+ $(LD) $(LDFLAGS) -N -e _start -Ttext 0 -o $U/initcode.out $U/initcode.o $U/usys.o $U/printf.o $(ULIB)  
$(OBJCOPY) -S -O binary $U/initcode.out $U/initcode  
$(OBJDUMP) -S $U/initcode.out > $U/initcode.asm
```

```
void  
main(void)  
{  
  static char buf[100];  
  
  // Read and run input commands.  
  while(getcmd(buf, sizeof(buf)) >= 0){  
    char *cmd = buf;  
    while (*cmd == ' ' || *cmd == '\t')  
      cmd++;  
    if (*cmd == '\n') // is a blank command  
      continue;  
    cmd[strlen(cmd) - 1] = '\0';  
    runcmd(cmd);  
  }  
}
```

用户态 Shell

.globl _start
.globl main
_start:
call main
stop:
j stop

strlen()
memset()
gets()

.global read
read:
li a7, SYS_read
ecall
ret
.global write
.....

vprintf()
fprintf()
printf()

```
void  
main(void)  
{
```

```
    static char buf[100];
```

```
    // Read and run input commands.
```

```
    while(getcmd(buf, sizeof(buf)) >= 0){
```

```
        char *cmd = buf;
```

```
        while (*cmd == ' ' || *cmd == '\t')
```

```
            cmd++;
```

```
        if (*cmd == '\n') // is a blank command
```

```
            continue;
```

```
        cmd[strlen(cmd) - 1] = '\0';
```

```
        runcmd(cmd);
```

```
    }
```

```
}
```

lib

```
$ make qemu  
qemu-system-riscv64 -machine virt -bios none -kernel  
kernel/kernel -m 128M -smp 3 -nographic
```

xv6 kernel is booting

hart 1 starting

hart 2 starting

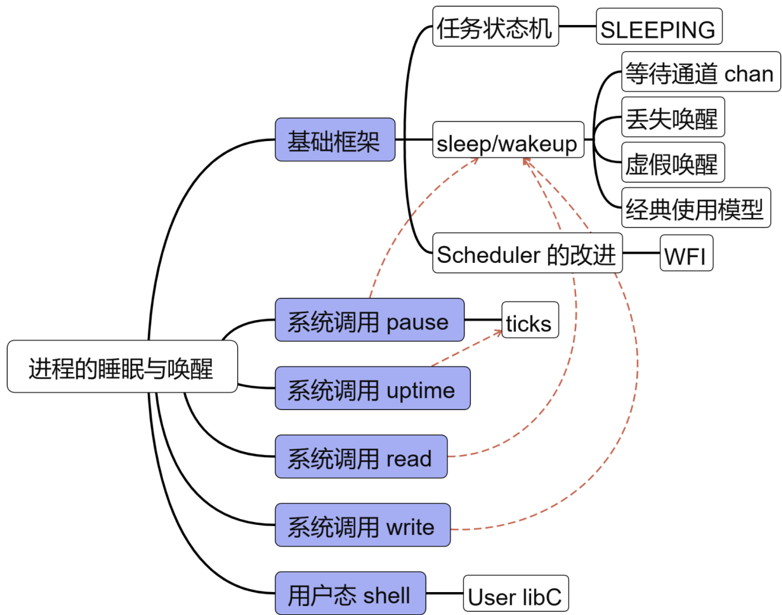
```
$ ls
```

```
Running command: 'ls', unsupported!
```

```
$
```



本章总结



谢谢

